Búsqueda en arreglos ordenados: comparación de B0, B1 y B2

Eduardo Alanis García

2026-11-11

Tabla de contenidos

1 Actividad: Algoritmos de búsqueda en el modelo de comparación **1**
1.1 Introducción . . . 1
1.2 Planteamiento del problema . . . 2
1.3 Desarrollo de las actividades . . . 2
1.3.1 Fundamento de los algoritmos . . . 2
1.3.2 Implementación . . . 2
1.3.3 Datos y caracterización de las instancias . . . 10
1.3.4 Medición . . . 13
1.3.5 Puntos de cruce . . . 20
1.3.6 Discusión de los resultados . . . 22
1.4 Conclusiones . . . 22
1.5 Referencias . . . 22

1. Actividad: Algoritmos de búsqueda en el modelo de comparación

1.1. Introducción

En esta actividad comparo cuatro algoritmos para encontrar la posición de inserción de un valor en un arreglo ordenado: búsqueda binaria acotada, búsqueda secuencial B0, búsqueda no acotada B1 y búsqueda no acotada B2. Además del tiempo de ejecución.

Para los experimentos utilizo un arreglo de 2^{20} enteros distintos y tres generadores de consultas: uniforme, en ráfagas y de cola pesada. Los generadores y sus semillas fijas son los de las notas. Para cada distribución analizo lotes de $m \in \{2^4, 2^6, \ldots, 2^{20}\}$ consultas.

1.2. Planteamiento del problema

Sea $A[1..n]$ un arreglo ascendente con elementos distintos y sea x el valor que se busca. El resultado es la posición de inserción p:

- Si $p = 1$, entonces $x \leq A[1]$.
- Si $2 \leq p \leq n$, entonces $A[p-1] < x \leq A[p]$.
- Si $p = n + 1$, entonces $A[n] < x$.

La tarea no consiste solamente en indicar si x aparece en el arreglo. Se debe encontrar la primera posición cuyo elemento sea mayor o igual que x. Si todos los elementos son menores que x, se devuelve n+1. Esta convención permite resolver de la misma manera las consultas que coinciden con un elemento y las que caen entre dos elementos.

1.3. Desarrollo de las actividades

1.3.1. Fundamento de los algoritmos

Las cotas presentadas en las notas se resumen en la siguiente tabla. Corresponden a una búsqueda con `sp=1`, por lo que en B0, B1 y B2, p es la posición de inserción desde el inicio; en la búsqueda binaria acotada, el costo se expresa en función del tamaño total n.

Algoritmo	Costo de comparaciones
Búsqueda binaria acotada	$O(\log n)$
Búsqueda secuencial B0	$O(\text{mín}(p, n))$
Búsqueda no acotada B1	$C_1(p) < 2\log_2 p + O(1)$
Búsqueda no acotada B2	$C_2(p) < \log_2 p + 2\log_2 \log_2 p + O(1)$

1.3.2. Implementación

1.3.2.1. Generación de datos sinteticos para la actividad

Todas las funciones usan semillas fijas.

```
using Random
using BenchmarkTools
using DataFrames
using Plots
using LaTeXStrings
using Statistics

gr()
default(fmt = :png)
```

```julia
const SEMILLA = 2026

"""
    genera_arreglo(n; g=8, semilla=SEMILLA)

Arreglo ordenado de n enteros distintos, acumulando huecos uniformes
en 1:g.
"""
function genera_arreglo(n; g=8, semilla=SEMILLA)
    rng = Xoshiro(semilla)
    cumsum(rand(rng, 1:g, n))
end

"""
    consultas_uniformes(U, m; semilla=SEMILLA + 1)

m valores distintos tomados uniformemente del universo 1:U, ordenados.
"""
function consultas_uniformes(U, m; semilla=SEMILLA + 1)
    rng = Xoshiro(semilla)
    sort!(randperm(rng, U)[1:m])
end

"""
    consultas_rafagas(U, m; k=8, semilla=SEMILLA + 2)

Consultas repartidas en k ráfagas. Cada ráfaga toma m ÷ k valores
distintos de una ventana de tamaño 4(m ÷ k).
"""
function consultas_rafagas(U, m; k=8, semilla=SEMILLA + 2)
    rng = Xoshiro(semilla)
    mk = m ÷ k
    w = 4mk
    X = Int[]

    for c in rand(rng, 1:U-w, k)
        append!(X, c .+ randperm(rng, w)[1:mk])
    end

    return sort!(unique!(X))
end
```

```
"""
    consultas_cola_pesada(U, m;  =1.5, semilla=SEMILLA + 3)

Genera huecos con distribución de cola pesada y los reescala al universo 1:U.
"""
function consultas_cola_pesada(U, m;  =1.5, semilla=SEMILLA + 3)
    rng = Xoshiro(semilla)
    h = rand(rng, m) .^ (-1 /  )
    c = cumsum(h)
    return unique!(ceil.(Int, U .* c ./ c[end]))
end

"""
    distancias(A, X)

Diferencias entre las posiciones de inserción de consultas consecutivas.
"""
function distancias(A, X)
    posiciones = searchsortedfirst.(Ref(A), X)
    return diff(posiciones)
end

cuantil(P, q) = sort(P)[max(1, round(Int, q * length(P)))]
```

```
cuantil (generic function with 1 method)
```

Salidas de control control: antes de medir nada, reproducir los valores para verificar que genero las mismas instancias.

```
n = 2^20
A = genera_arreglo(n)
U = A[end]

generadores = (
    ("Uniforme", consultas_uniformes),
    ("Ráfagas", consultas_rafagas),
    ("Cola pesada", consultas_cola_pesada)
)

@assert length(A) == n
@assert issorted(A)
```

```
@assert all(diff(A) .> 0)
@assert U >= 2^20

println("n = $(length(A)), U = $U")
```

```
n = 1048576, U = 4721224
```

```
generadores_control = (
    ("uniforme", consultas_uniformes),
    ("ráfagas", consultas_rafagas),
    ("cola pesada", consultas_cola_pesada)
)

for (nombre, generar) in generadores_control, m in (2^8, 2^14, 2^20)
    X = generar(U, m)
    P = distancias(A, X)
    etiqueta = rpad(nombre, 12) * rpad("m = 2^$(Int(log2(m)))", 10)
    etiqueta *= rpad("|X| = $(length(X))", 16)
    q = (cuantil(P, 0.5), cuantil(P, 0.9), cuantil(P, 0.99), maximum(P))
    println(etiqueta, "p: mediana = $(q[1]), q90 = $(q[2]), q99 = $(q[3]), máx = $(q[4])")
end
```

```
uniforme    m = 2^8   |X| = 256       p: mediana = 2715, q90 = 9422, q99 = 17499, máx = 20902
uniforme    m = 2^14  |X| = 16384     p: mediana = 44, q90 = 150, q99 = 295, máx = 633
uniforme    m = 2^20  |X| = 1048576   p: mediana = 1, q90 = 2, q99 = 5, máx = 15
ráfagas     m = 2^8   |X| = 256       p: mediana = 1, q90 = 3, q99 = 131250, máx = 176250
ráfagas     m = 2^14  |X| = 16384     p: mediana = 1, q90 = 2, q99 = 4, máx = 174156
ráfagas     m = 2^20  |X| = 1028146   p: mediana = 1, q90 = 2, q99 = 4, máx = 40102
cola pesada m = 2^8   |X| = 256       p: mediana = 2698, q90 = 7353, q99 = 18519, máx = 48708
cola pesada m = 2^14  |X| = 16384     p: mediana = 33, q90 = 96, q99 = 416, máx = 32400
cola pesada m = 2^20  |X| = 1048576   p: mediana = 1, q90 = 2, q99 = 7, máx = 5976
```

1.3.2.2. Implementación de los algoritmos

```
"""
    seqsearch(A, x, sp=1)

Búsqueda unaria B0 desde la pista `sp`.
"""
function seqsearch(A, x, sp=1)
```

```
    n = length(A)
    while sp <= n && x > A[sp]
        sp += 1
    end

    return sp
end

"""
    binarysearch(A, x, sp=1, ep=length(A))

Encuentra la posición de inserción de `x` en `A`, suponiendo que está en
`sp:ep+1`.
"""
function binarysearch(A, x, sp=1, ep=length(A))
    lo, hi = sp, ep + 1
    while lo < hi
        mid = (lo + hi) ÷ 2
        if x <= A[mid]
            hi = mid
        else
            lo = mid + 1
        end
    end

    return lo
end

"""
    doublingsearch(A, x, sp=1)

Búsqueda doblada B1; los sondeos se miden como desplazamientos desde `sp`.
"""
function doublingsearch(A, x, sp=1)
    n = length(A)
    p = 0
    i = 1

    while sp+i <= n && A[sp+i] < x
        p = i
        i += i
    end
```

```
    return binarysearch(A, x, sp + p, min(n, sp+i))
end
```

```
doublingsearch
```

Búsqueda doblemente doblada B2 de las notas.

```
function doublingdoublingsearch(A, x, sp=1)
    n = length(A)
    pos(e) = e >= 62 ? typemax(Int) : sp + (1 << e)
    reached(e) = pos(e) > n || x <= A[pos(e)]

    lo, hi = -1, 1
    while !reached(hi)
        lo = hi
        hi += hi
    end

    lo += 1
    while lo < hi
        mid = (lo + hi) ÷ 2
        if reached(mid)
            hi = mid
        else
            lo = mid + 1
        end
    end

    first = hi == 0 ? sp : pos(hi - 1)
    return binarysearch(A, x, first, min(n, pos(hi)))
end
```

```
doublingdoublingsearch (generic function with 2 methods)
```

`CountingArray` y `comparaciones` se incorporan tal como aparecen en el listado 3 de las notas.

```
mutable struct CountingArray{T} <: AbstractVector{T}
    data::Vector{T}
    count::Int
```

```
end

CountingArray(data) = CountingArray(data, 0)
Base.size(C::CountingArray) = size(C.data)
Base.IndexStyle(::Type{<:CountingArray}) = IndexLinear()

function Base.getindex(C::CountingArray, i::Int)
    C.count += 1
    C.data[i]
end

"""
    comparaciones(f, A, x, args...)

Ejecuta la búsqueda `f(A, x, args...)` y regresa la tupla (posición, número de comparaciones)
"""
function comparaciones(f, A::CountingArray, x, args...)
    A.count = 0
    p = f(A, x, args...)
    (p, A.count)
end
```

```
comparaciones
```

1.3.2.3. Veficación de los algoritmos contra `searchsortedfirst`

Verifico que las cuatro búsquedas coincidan con `searchsortedfirst` sobre arreglos aleatorios y con pistas válidas.

```
algoritmos = (
    ("Búsqueda binaria", binarysearch),
    ("Búsqueda secuencial B0", seqsearch),
    ("Búsqueda no acotada B1", doublingsearch),
    ("Búsqueda no acotada B2", doublingdoublingsearch)
)

rng = Xoshiro(SEMILLA + 10)

# Probar diferentes tamaños de arreglos, con y sin duplicados
for n in (1, 2, 7, 31, 128), con_duplicados in (false, true)

    # Arreglo aleatorio ordenado
```

```
    arreglo = con_duplicados ?
        sort!(rand(rng, 1:4*n, n)) :
        sort!(randperm(rng, 4*n)[1:n])

    # Asegurar que existan valores repetidos cuando corresponda
    if con_duplicados && n > 1
        arreglo[end] = arreglo[1]
        sort!(arreglo)
    end

    # Crear los valores que se van a buscar.
    consultas = vcat(
        [0, 4*n + 1],
        arreglo,
        arreglo .- 1,
        arreglo .+ 1,
        rand(rng, 0:4*n+1, 4*n)
    )

    sort!(unique!(consultas))

    # Probar cada valor de búsqueda.
    for x in consultas

        # Obtener la posición esperada.
        posicion_esperada = searchsortedfirst(arreglo, x)

        # Probar diferentes pistas.
        pistas = unique((
            1,
            cld(posicion_esperada, 2),
            posicion_esperada
        ))

        # Comparar los resultados de los cuatro algoritmos.
        for (_, algoritmo) in algoritmos, sp in pistas
            @assert algoritmo(arreglo, x, sp) == posicion_esperada
        end
    end
end

println("Las cuatro búsquedas coinciden con searchsortedfirst.")
```

```
Las cuatro búsquedas coinciden con searchsortedfirst.
```

1.3.2.4. Función buscar_todas

Escriba una función buscar_todas(f, A, X) que resuelva en orden todas las consultas de X con el algoritmo f, comenzando cada búsqueda donde terminó la anterior; para la búsqueda binaria independiente la pista será siempre 1. Puede usar CountingArray para contar comparaciones.

```
function buscar_todas(algoritmo, A, X)
    posiciones = Vector{Int}(undef, length(X)) # Vector para almacenar las posiciones de inse
    sp = 1

    for (j, i) in enumerate(eachindex(X))
        pista = algoritmo === binarysearch ? 1 : sp # Para búsqueda binaria independiente, la
        p = algoritmo(A, X[i], pista)
        posiciones[j] = p
        sp = p                                  # Actualizar para la siguiente consulta
    end

    return posiciones
end

function contar_todas(algoritmo, A::CountingArray, X)
    A.count = 0
    posiciones = buscar_todas(algoritmo, A, X)
    return posiciones, A.count
end
```

```
contar_todas (generic function with 1 method)
```

1.3.3. Datos y caracterización de las instancias

Genere el arreglo A = genera_arreglo(2^20) y, para cada $m \in \{2^4, 2^6, \ldots, 2^{20}\}$, genere lotes de consultas con cada uno de los tres generadores. Antes de medir, calcule para cada configuración las distancias entre respuestas consecutivas con distancias y muestre su distribución, e.g., con histogramas en escala logarítmica o con los cuantiles 0.5, 0.9 y 0.99. Compare estas distancias con los umbrales $\sqrt{}$ y /log2 2 del capítulo.

```
n = 2^20
A = genera_arreglo(n)
U = A[end]
```

```
ms = 2 .^ (4:2:20)
A_contada = CountingArray(A)

perfil_instancias = DataFrame(
    distribucion = String[],
    m_solicitado = Int[],
    mediana_distancia = Int[],
    q90_distancia = Int[],
    q99_distancia = Int[],
    maxima_distancia = Int[],
    fraccion_menor_raiz_n = Float64[],
    fraccion_menor_n_sobre_log2_n2 = Float64[]
)

lotes = Dict{Tuple{String, Int}, Vector{Int}}()

for (distribucion, generar) in generadores, m in ms
    X = generar(U, m)
    @assert issorted(X)
    @assert !isempty(X)
    lotes[(distribucion, m)] = X

    P = distancias(A, X)
    raiz_n = sqrt(n)
    umbral_b2 = n / log2(n)^2
    push!(perfil_instancias, (
        distribucion,
        m,
        cuantil(P, 0.50),
        cuantil(P, 0.90),
        cuantil(P, 0.99),
        maximum(P),
        count(p -> p < raiz_n, P) / length(P),   # Proporción de distancias menores que √n
        count(p -> p < umbral_b2, P) / length(P) # Proporción de distancias menores que n/log
    ))
end

rename!(perfil_instancias,
    :distribucion => "Distribución",
    :m_solicitado => "m solicitado",
    :mediana_distancia => "Mediana de p",
    :q90_distancia => "Cuantil 0.90 de p",
```

```
    :q99_distancia => "Cuantil 0.99 de p",
    :maxima_distancia => "Máximo de p",
    :fraccion_menor_raiz_n => "Fracción p < √n",
    :fraccion_menor_n_sobre_log2_n2 => "Fracción p < n/log ²(n)"
)

PrettyTables.pretty_table(
    Markdown.HTML,
    perfil_instancias;
    backend = :html,
    limit_printing = false,
    maximum_number_of_rows = -1,
    maximum_number_of_columns = -1
)
```

Distribución String	m solicitado Int64	Mediana de p Int64	Cuantil 0.90 de p Int64	Cuantil 0.99 de p Int64	Máximo de p Int64	Fracción p < √n Float64	Fracción p < n/log ²(n) Float64
Uniforme	16	42147	149974	290898	290898	0.0	0.0
Uniforme	64	9752	38120	94007	125420	0.0793651	0.15873
Uniforme	256	2715	9422	17499	20902	0.211765	0.486275
Uniforme	1024	685	2398	4967	8343	0.642229	0.916911
Uniforme	4096	180	581	1195	2229	0.979243	1.0
Uniforme	16384	44	150	295	633	1.0	1.0
Uniforme	65536	11	37	73	167	1.0	1.0
Uniforme	262144	3	9	19	47	1.0	1.0
Uniforme	1048576	1	2	5	15	1.0	1.0
Ráfagas	16	1	170993	176283	176283	0.533333	0.533333
Ráfagas	64	1	59275	170988	176277	0.888889	0.888889
Ráfagas	256	1	3	131250	176250	0.972549	0.972549
Ráfagas	1024	1	2	4	176144	0.993157	0.993157
Ráfagas	4096	1	2	4	175743	0.998291	0.998291
Ráfagas	16384	1	2	4	174156	0.999573	0.999573
Ráfagas	65536	1	2	4	167731	0.999893	0.999893
Ráfagas	262144	1	2	4	142195	0.999973	0.999973
Ráfagas	1048576	1	2	4	40102	0.999997	0.999998
Cola pesada	16	35883	148643	164095	164095	0.0	0.0
Cola pesada	64	8875	33149	93479	140683	0.0	0.0
Cola pesada	256	2698	7353	18519	48708	0.0	0.486275
Cola pesada	1024	625	1883	6110	14366	0.770283	0.944282
Cola pesada	4096	123	368	1572	94076	0.979243	0.996337
Cola pesada	16384	33	96	416	32400	0.997131	0.998962
Cola pesada	65536	9	25	111	20292	0.999466	0.999893
Cola pesada	262144	2	6	28	23601	0.99995	0.999985
Cola pesada	1048576	1	2	7	5976	0.999991	0.999997

La distancia `p` de esta tabla es el desplazamiento entre las posiciones de inserción de dos consultas consecutivas. Las dos últimas columnas muestran qué fracción de las distancias queda bajo los umbrales teóricos para B1 y B2.

1.3.4. Medición

Para cada algoritmo, tamaño y distribución, mida el número total de comparaciones y el tiempo acumulado para resolver las consultas; repita las mediciones de tiempo (e.g., con @benchmark) para obtener valores fiables.

```
resultados = DataFrame(
    distribucion = String[],
    n = Int[],
    m_solicitado = Int[],
    algoritmo = String[],
    comparaciones_totales = Int[],
    comparaciones_promedio = Float64[],
    tiempo_total_ns = Float64[],
    tiempo_por_consulta_ns = Float64[],
    muestras = Int[]
)

for (distribucion, generar) in generadores, m in ms
    X = lotes[(distribucion, m)]
    consultas_esperadas = searchsortedfirst.(Ref(A), X)

    for (nombre, algoritmo) in algoritmos
        posiciones, comparaciones_totales = contar_todas(algoritmo, A_contada, X)
        @assert posiciones == consultas_esperadas

        trial = @benchmark(
            buscar_todas($algoritmo, $A, $X),
            samples = 10,
            evals = 1
        )

        tiempo = median(trial).time

        push!(resultados, (
            distribucion,
            n,
            m,
            nombre,
            comparaciones_totales,
            comparaciones_totales / length(X),
            tiempo,
            tiempo / length(X),
            length(trial)
        ))
    end
end
```

```
PrettyTables.pretty_table(
    Markdown.HTML,
    resultados;
    backend = :html,
    limit_printing = false,
    maximum_number_of_rows = -1,
    maximum_number_of_columns = -1
)
```

distribucion String	n Int64	m_solicitado Int64	algoritmo String	comparaciones_totales Int64	comparaciones_promedio Float64	tiempo_total_ns Float64	tiempo_por
Uniforme	1048576	16	Búsqueda binaria	320	20.0	2050.0	
Uniforme	1048576	16	Búsqueda secuencial B0	1011473	63217.1	1.4524e6	
Uniforme	1048576	16	Búsqueda no acotada B1	494	30.875	650.0	
Uniforme	1048576	16	Búsqueda no acotada B2	353	22.0625	1200.0	
Uniforme	1048576	64	Búsqueda binaria	1280	20.0	3550.0	
Uniforme	1048576	64	Búsqueda secuencial B0	1048105	16376.6	504650.0	
Uniforme	1048576	64	Búsqueda no acotada B1	1705	26.6406	4100.0	
Uniforme	1048576	64	Búsqueda no acotada B2	1283	20.0469	3000.0	
Uniforme	1048576	256	Búsqueda binaria	5120	20.0	41550.0	
Uniforme	1048576	256	Búsqueda secuencial B0	1048297	4094.91	583550.0	
Uniforme	1048576	256	Búsqueda no acotada B1	5944	23.2188	22850.0	
Uniforme	1048576	256	Búsqueda no acotada B2	4704	18.375	12450.0	
Uniforme	1048576	1024	Búsqueda binaria	20480	20.0	195000.0	
Uniforme	1048576	1024	Búsqueda secuencial B0	1049508	1024.91	751450.0	
Uniforme	1048576	1024	Búsqueda no acotada B1	19770	19.3066	152400.0	
Uniforme	1048576	1024	Búsqueda no acotada B2	16525	16.1377	132900.0	
Uniforme	1048576	4096	Búsqueda binaria	81920	20.0	553700.0	
Uniforme	1048576	4096	Búsqueda secuencial B0	1052580	256.978	687500.0	
Uniforme	1048576	4096	Búsqueda no acotada B1	63341	15.4641	427100.0	
Uniforme	1048576	4096	Búsqueda no acotada B2	54616	13.334	865200.0	
Uniforme	1048576	16384	Búsqueda binaria	327680	20.0	1.76535e6	
Uniforme	1048576	16384	Búsqueda secuencial B0	1064868	64.9944	796950.0	
Uniforme	1048576	16384	Búsqueda no acotada B1	189053	11.5389	1.0499e6	
Uniforme	1048576	16384	Búsqueda no acotada B2	169650	10.3546	1.5172e6	
Uniforme	1048576	65536	Búsqueda binaria	1310720	20.0	6.801e6	
Uniforme	1048576	65536	Búsqueda secuencial B0	1114088	16.9996	1.85e6	
Uniforme	1048576	65536	Búsqueda no acotada B1	523280	7.98462	4.09315e6	
Uniforme	1048576	65536	Búsqueda no acotada B2	497687	7.5941	3.4924e6	
Uniforme	1048576	262144	Búsqueda binaria	5242880	20.0	2.07102e7	
Uniforme	1048576	262144	Búsqueda secuencial B0	1310719	5.0	5.02865e6	
Uniforme	1048576	262144	Búsqueda no acotada B1	1315838	5.01952	6.1885e6	
Uniforme	1048576	262144	Búsqueda no acotada B2	1315859	5.0196	8.0658e6	
Uniforme	1048576	1048576	Búsqueda binaria	20971522	20.0	8.54117e7	
Uniforme	1048576	1048576	Búsqueda secuencial B0	2097151	2.0	1.34783e7	
Uniforme	1048576	1048576	Búsqueda no acotada B1	3510472	3.34785	1.76984e7	
Uniforme	1048576	1048576	Búsqueda no acotada B2	4231768	4.03573	2.35602e7	
Ráfagas	1048576	16	Búsqueda binaria	320	20.0	1500.0	
Ráfagas	1048576	16	Búsqueda secuencial B0	986936	61683.5	524250.0	
Ráfagas	1048576	16	Búsqueda no acotada B1	300	18.75	400.0	
Ráfagas	1048576	16	Búsqueda no acotada B2	219	13.6875	600.0	
Ráfagas	1048576	64	Búsqueda binaria	1280	20.0	2900.0	
Ráfagas	1048576	64	Búsqueda secuencial B0	986984	15421.6	552850.0	
Ráfagas	1048576	64	Búsqueda no acotada B1	459	7.17188	1200.0	

distribucion String	n Int64	m_solicitado Int64	algoritmo String	comparaciones_totales Int64	comparaciones_promedio Float64	tiempo_total_ns Float64	tiempo_por
Ráfagas	1048576	64	Búsqueda no acotada B2	411	6.42188	1050.0	
Ráfagas	1048576	256	Búsqueda binaria	5120	20.0	12400.0	
Ráfagas	1048576	256	Búsqueda secuencial B0	987178	3856.16	547400.0	
Ráfagas	1048576	256	Búsqueda no acotada B1	1101	4.30078	4150.0	
Ráfagas	1048576	256	Búsqueda no acotada B2	1185	4.62891	5050.0	
Ráfagas	1048576	1024	Búsqueda binaria	20480	20.0	98700.0	
Ráfagas	1048576	1024	Búsqueda secuencial B0	987950	964.795	1.18805e6	
Ráfagas	1048576	1024	Búsqueda no acotada B1	3609	3.52441	8250.0	
Ráfagas	1048576	1024	Búsqueda no acotada B2	4257	4.15723	21700.0	
Ráfagas	1048576	4096	Búsqueda binaria	81920	20.0	508600.0	
Ráfagas	1048576	4096	Búsqueda secuencial B0	991041	241.953	608950.0	
Ráfagas	1048576	4096	Búsqueda no acotada B1	13719	3.34937	91800.0	
Ráfagas	1048576	4096	Búsqueda no acotada B2	16596	4.05176	72000.0	
Ráfagas	1048576	16384	Búsqueda binaria	327680	20.0	1.2733e6	
Ráfagas	1048576	16384	Búsqueda secuencial B0	1003412	61.2434	703550.0	
Ráfagas	1048576	16384	Búsqueda no acotada B1	54092	3.30151	238500.0	
Ráfagas	1048576	16384	Búsqueda no acotada B2	66049	4.03131	268100.0	
Ráfagas	1048576	65536	Búsqueda binaria	1310720	20.0	4.90595e6	
Ráfagas	1048576	65536	Búsqueda secuencial B0	1052865	16.0654	1.41275e6	
Ráfagas	1048576	65536	Búsqueda no acotada B1	215213	3.28389	912650.0	
Ráfagas	1048576	65536	Búsqueda no acotada B2	263616	4.02246	1.20435e6	
Ráfagas	1048576	262144	Búsqueda binaria	5242880	20.0	2.0735e7	
Ráfagas	1048576	262144	Búsqueda secuencial B0	1250768	4.7713	6.1647e6	
Ráfagas	1048576	262144	Búsqueda no acotada B1	859743	3.27966	3.65385e6	
Ráfagas	1048576	262144	Búsqueda no acotada B2	1054113	4.02112	4.65825e6	
Ráfagas	1048576	1048576	Búsqueda binaria	20562920	20.0	7.54436e7	
Ráfagas	1048576	1048576	Búsqueda secuencial B0	2021880	1.96653	1.3336e7	
Ráfagas	1048576	1048576	Búsqueda no acotada B1	3343144	3.25162	1.58734e7	
Ráfagas	1048576	1048576	Búsqueda no acotada B2	4132056	4.01894	2.3607e7	
Cola pesada	1048576	16	Búsqueda binaria	320	20.0	1750.0	
Cola pesada	1048576	16	Búsqueda secuencial B0	1048591	65536.9	575900.0	
Cola pesada	1048576	16	Búsqueda no acotada B1	515	32.1875	700.0	
Cola pesada	1048576	16	Búsqueda no acotada B2	362	22.625	1100.0	
Cola pesada	1048576	64	Búsqueda binaria	1280	20.0	7400.0	
Cola pesada	1048576	64	Búsqueda secuencial B0	1048639	16385.0	584600.0	
Cola pesada	1048576	64	Búsqueda no acotada B1	1783	27.8594	3100.0	
Cola pesada	1048576	64	Búsqueda no acotada B2	1330	20.7812	2850.0	
Cola pesada	1048576	256	Búsqueda binaria	5120	20.0	55700.0	
Cola pesada	1048576	256	Búsqueda secuencial B0	1048831	4097.0	1.1663e6	
Cola pesada	1048576	256	Búsqueda no acotada B1	6184	24.1562	13500.0	
Cola pesada	1048576	256	Búsqueda no acotada B2	4866	19.0078	21550.0	
Cola pesada	1048576	1024	Búsqueda binaria	20480	20.0	213650.0	
Cola pesada	1048576	1024	Búsqueda secuencial B0	1049599	1025.0	579050.0	
Cola pesada	1048576	1024	Búsqueda no acotada B1	20562	20.0801	138800.0	
Cola pesada	1048576	1024	Búsqueda no acotada B2	17372	16.9648	149250.0	
Cola pesada	1048576	4096	Búsqueda binaria	81920	20.0	572850.0	
Cola pesada	1048576	4096	Búsqueda secuencial B0	1052671	257.0	1.3071e6	
Cola pesada	1048576	4096	Búsqueda no acotada B1	63498	15.5024	322700.0	
Cola pesada	1048576	4096	Búsqueda no acotada B2	53626	13.0923	481300.0	
Cola pesada	1048576	16384	Búsqueda binaria	327680	20.0	1.74185e6	
Cola pesada	1048576	16384	Búsqueda secuencial B0	1064959	64.9999	1.31185e6	
Cola pesada	1048576	16384	Búsqueda no acotada B1	191627	11.696	1.24665e6	
Cola pesada	1048576	16384	Búsqueda no acotada B2	179443	10.9523	1.026e6	
Cola pesada	1048576	65536	Búsqueda binaria	1310720	20.0	5.6314e6	
Cola pesada	1048576	65536	Búsqueda secuencial B0	1114111	17.0	1.5572e6	
Cola pesada	1048576	65536	Búsqueda no acotada B1	527119	8.0432	2.14525e6	
Cola pesada	1048576	65536	Búsqueda no acotada B2	496734	7.57956	3.1677e6	
Cola pesada	1048576	262144	Búsqueda binaria	5242880	20.0	2.55104e7	

distribucion String	n Int64	m_solicitado Int64	algoritmo String	comparaciones_totales Int64	comparaciones_promedio Float64	tiempo_total_ns Float64	tiempo_por
Cola pesada	1048576	262144	Búsqueda secuencial B0	1310719	5.0	4.5405e6	
Cola pesada	1048576	262144	Búsqueda no acotada B1	1212212	4.62422	5.4391e6	
Cola pesada	1048576	262144	Búsqueda no acotada B2	1203588	4.59132	7.265e6	
Cola pesada	1048576	1048576	Búsqueda binaria	20971521	20.0	9.10596e7	
Cola pesada	1048576	1048576	Búsqueda secuencial B0	2097151	2.0	1.3642e7	
Cola pesada	1048576	1048576	Búsqueda no acotada B1	3428118	3.26931	1.51119e7	
Cola pesada	1048576	1048576	Búsqueda no acotada B2	4272816	4.07487	1.85972e7	

La tabla presenta comparaciones y tiempo acumulado para cada distribución, tamaño y algoritmo.

1.3.4.1. Reporte comparaciones en graficas

Reporte comparaciones y tiempo por separado, en gráficas con m en el eje horizontal (escala logarítmica) y una curva por algoritmo, una gráfica por distribución.

```
nombre_distribuciones = [nombre for (nombre, _) in generadores]
nombre_algoritmos = [nombre for (nombre, _) in algoritmos]
graficas = Any[]

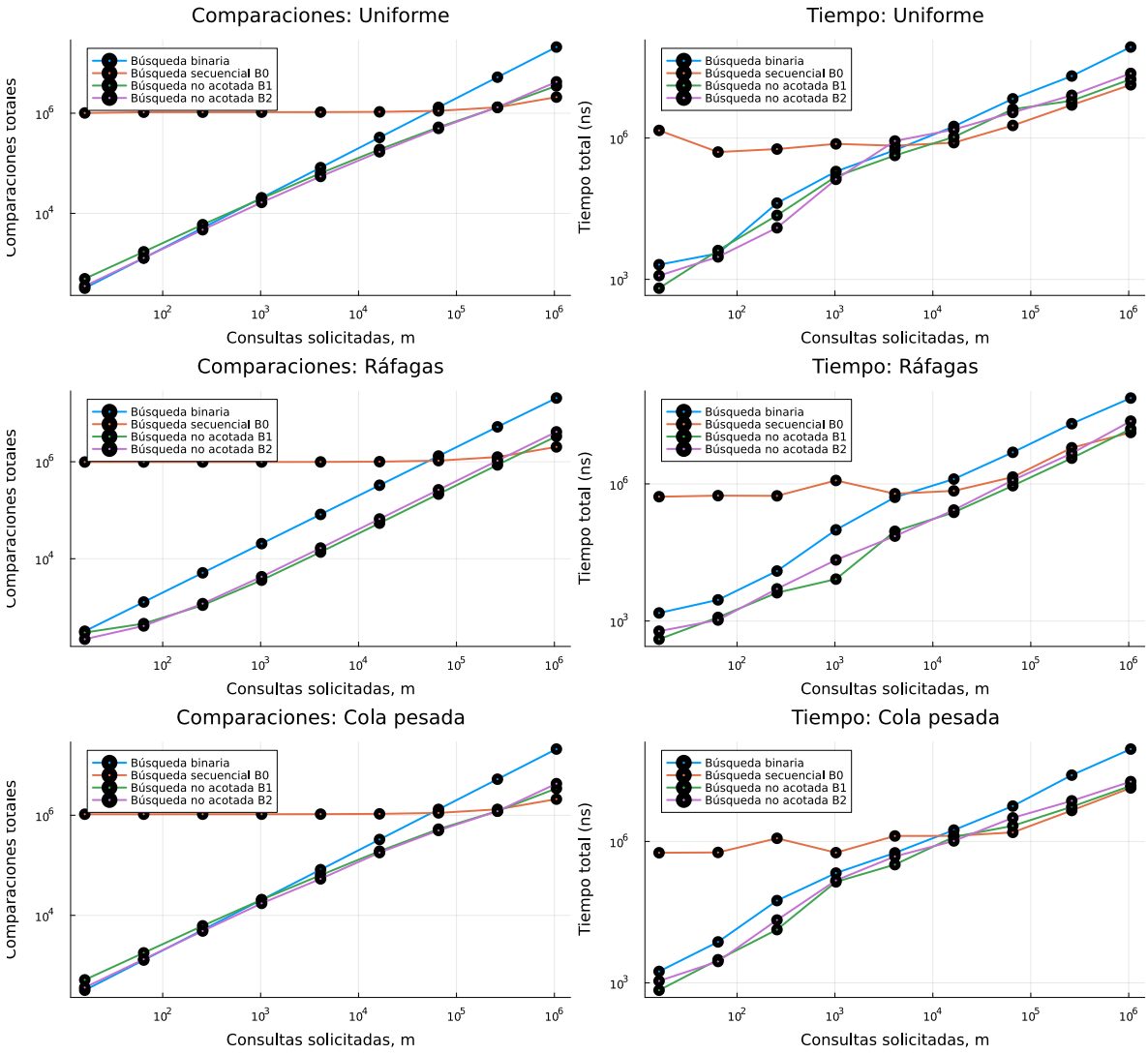
for distribucion in nombre_distribuciones
    datos = resultados[resultados.distribucion .== distribucion, :]
    figura_comparaciones = plot(
        xlabel = "Consultas solicitadas, m",
        ylabel = "Comparaciones totales",
        title = "Comparaciones: $distribucion",
        xscale = :log10,
        yscale = :log10,
        legend = :topleft
    )
    figura_tiempos = plot(
        xlabel = "Consultas solicitadas, m",
        ylabel = "Tiempo total (ns)",
        title = "Tiempo: $distribucion",
        xscale = :log10,
        yscale = :log10,
        legend = :topleft
    )

    for nombre in nombre_algoritmos
        datos_algoritmo = sort(
            datos[datos.algoritmo .== nombre, :],
            :m_solicitado
        )
        plot!(
            figura_comparaciones,
            datos_algoritmo.m_solicitado,
            datos_algoritmo.comparaciones_totales,
            label = nombre,
            marker = :circle,
```

```
            linewidth = 2
        )
        plot!(
            figura_tiempos,
            datos_algoritmo.m_solicitado,
            datos_algoritmo.tiempo_total_ns,
            label = nombre,
            marker = :circle,
            linewidth = 2
        )
    end

    push!(graficas, figura_comparaciones, figura_tiempos)
end

plot(graficas..., layout = (3, 2), size = (1200, 1100))
```



1.3.5. Puntos de cruce

El siguiente resumen localiza, para cada distribución, el primer tamaño medido donde B1 o B2 deja de usar menos comparaciones que la búsqueda binaria independiente, y donde B0 iguala o mejora ese costo. Para el tiempo se aplica el mismo criterio a la mediana del tiempo total.

```
function primer_cruce(datos, distribucion, algoritmo, columna; b0=false)
    comparacion = sort(
        datos[datos.distribucion .== distribucion, :],
        :m_solicitado
    )
```

```
    base = comparacion[comparacion.algoritmo .== "Búsqueda binaria", :]
    metodo = comparacion[comparacion.algoritmo .== algoritmo, :]
    condicion(i) = b0 ? metodo[i, columna] <= base[i, columna] :
                        metodo[i, columna] >= base[i, columna]
    indice = findfirst(condicion, eachindex(metodo.m_solicitado))

    if isnothing(indice)
        return "Sin cruce hasta m=$(maximum(metodo.m_solicitado))"
    end

    m = metodo.m_solicitado[indice]
    return "m=$m (m/n=$(round(m / metodo.n[indice], digits=4)))"
end

tabla_cruces = DataFrame(
    distribucion = String[],
    metrica = String[],
    algoritmo = String[],
    primer_cruce = String[]
)

for distribucion in nombre_distribuciones
    for (algoritmo, es_b0) in (
        ("Búsqueda secuencial B0", true),
        ("Búsqueda no acotada B1", false),
        ("Búsqueda no acotada B2", false)
    )
        for (metrica, columna) in (
            ("Comparaciones", :comparaciones_totales),
            ("Tiempo", :tiempo_total_ns)
        )
            push!(tabla_cruces, (
                distribucion,
                metrica,
                algoritmo,
                primer_cruce(resultados, distribucion, algoritmo, columna; b0=es_b0)
            ))
        end
    end
end

rename!(tabla_cruces,
```

```
    :distribucion => "Distribución",
    :metrica => "Métrica",
    :algoritmo => "Algoritmo",
    :primer_cruce => "Primer cruce observado"
)

PrettyTables.pretty_table(
    Markdown.HTML,
    tabla_cruces;
    backend = :html,
    limit_printing = false,
    maximum_number_of_rows = -1,
    maximum_number_of_columns = -1
)
```

Distribución String	Métrica String	Algoritmo String	Primer cruce observado String
Uniforme	Comparaciones	Búsqueda secuencial B0	m=65536 (m/n=0.0625)
Uniforme	Tiempo	Búsqueda secuencial B0	m=16384 (m/n=0.0156)
Uniforme	Comparaciones	Búsqueda no acotada B1	m=16 (m/n=0.0)
Uniforme	Tiempo	Búsqueda no acotada B1	m=64 (m/n=0.0001)
Uniforme	Comparaciones	Búsqueda no acotada B2	m=16 (m/n=0.0)
Uniforme	Tiempo	Búsqueda no acotada B2	m=4096 (m/n=0.0039)
Ráfagas	Comparaciones	Búsqueda secuencial B0	m=65536 (m/n=0.0625)
Ráfagas	Tiempo	Búsqueda secuencial B0	m=16384 (m/n=0.0156)
Ráfagas	Comparaciones	Búsqueda no acotada B1	Sin cruce hasta m=1048576
Ráfagas	Tiempo	Búsqueda no acotada B1	Sin cruce hasta m=1048576
Ráfagas	Comparaciones	Búsqueda no acotada B2	Sin cruce hasta m=1048576
Ráfagas	Tiempo	Búsqueda no acotada B2	Sin cruce hasta m=1048576
Cola pesada	Comparaciones	Búsqueda secuencial B0	m=65536 (m/n=0.0625)
Cola pesada	Tiempo	Búsqueda secuencial B0	m=16384 (m/n=0.0156)
Cola pesada	Comparaciones	Búsqueda no acotada B1	m=16 (m/n=0.0)
Cola pesada	Tiempo	Búsqueda no acotada B1	Sin cruce hasta m=1048576
Cola pesada	Comparaciones	Búsqueda no acotada B2	m=16 (m/n=0.0)
Cola pesada	Tiempo	Búsqueda no acotada B2	Sin cruce hasta m=1048576

1.3.6. Discusión de los resultados

1.4. Conclusiones

1.5. Referencias

Téllez, E. S. (s. f.-a). *Algoritmos de búsqueda en el modelo de comparación* (capítulo 5). Curso Introductorio al Análisis de Algoritmos con Julia. https://sadit.github.io/ALGO-IR/cap5-busqueda.html